

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

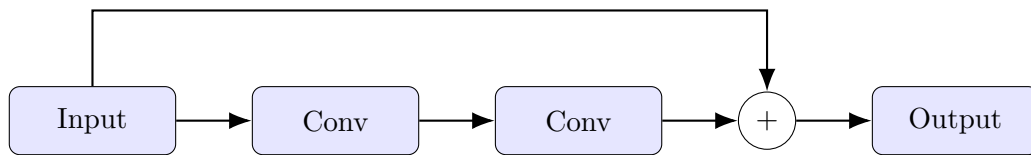


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

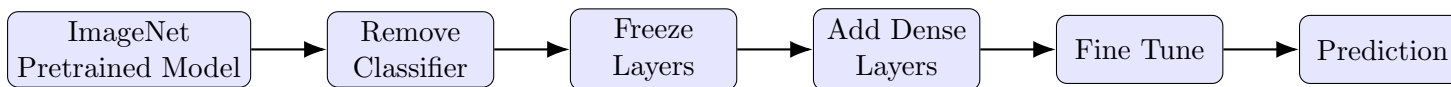


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.

4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Experimental Results and Observations

The experiments were carried out on the CIFAR-10 dataset using different CNN architectures. The models were trained and evaluated using the same dataset so that their classification performance could be compared directly. Training accuracy, validation accuracy, loss, test accuracy and other evaluation metrics were recorded for analysis.

16.1 Dataset Preparation

The CIFAR-10 dataset contains 50,000 training images and 10,000 testing images distributed among ten classes. Each image has a size of $32 \times 32 \times 3$. The pixel values were normalized to the range $[0, 1]$ before training.

Table 1: CIFAR-10 Dataset Details

Property	Value
Training images	50,000
Testing images	10,000
Number of classes	10
Image size	$32 \times 32 \times 3$
Pixel value range after normalization	0–1

Observation:

The dataset contains a good variety of object categories, but the small image resolution makes some classes difficult to distinguish. In particular, classes such as cat, dog, bird and deer contain visually similar features, which is reflected later in the class-wise classification results.

16.2 Sample Images



Figure 3: Sample images from the CIFAR-10 dataset

Observation:

The sample images represent the different object categories present in the dataset. Since the images are only 32×32 pixels, important details such as object shape and texture are limited, making the classification task more challenging.

16.3 LeNet-5

The implemented LeNet-5 model contained 136,886 trainable parameters. The model was trained for 10 epochs using Adam with a learning rate of 0.001 and a batch size of 32.

Table 2: LeNet-5 Training and Evaluation Results

Metric	Value
Total Parameters	136,886
Training Epochs	10
Best Validation Accuracy	64.22%
Test Accuracy	62.04%
Training Time	60.56 s

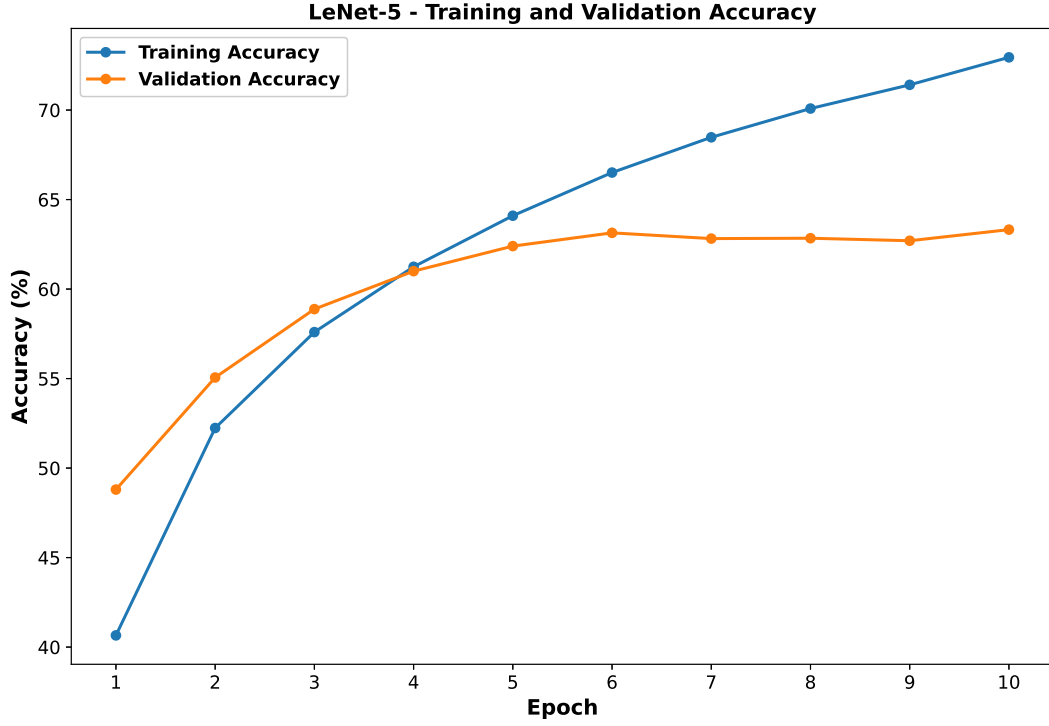


Figure 4: Training and validation accuracy of LeNet-5

Observation:

The training accuracy increased consistently over the 10 epochs and reached 72.59% by the end. The validation accuracy improved as well, but at a slower rate, with the highest value reaching 64.22%. This shows that LeNet-5 was able to learn from the training images, but its performance on unseen CIFAR-10 images was comparatively limited.

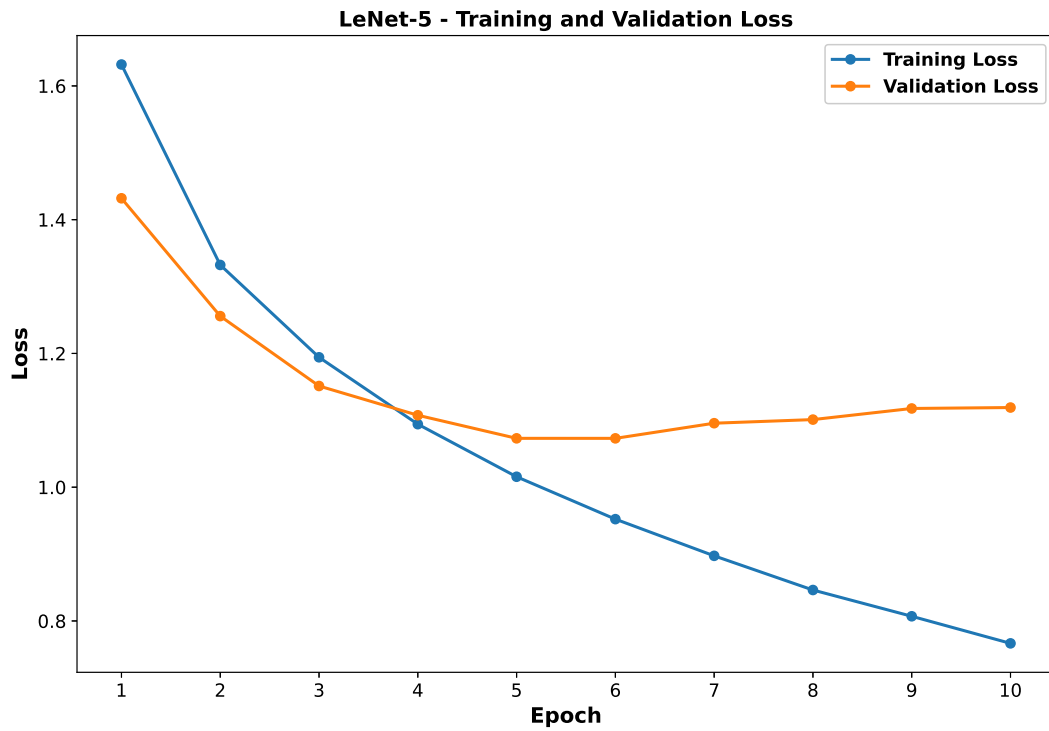


Figure 5: Training and validation loss of LeNet-5

Observation:

The training loss kept decreasing from 1.5944 in the first epoch to 0.7799 in the last epoch, so the model was learning properly from the training data. The validation loss also decreased at first, but after reaching its lowest point around the middle of training, it started increasing slightly. This shows that training for more epochs did not improve the model much on the validation data.

16.4 AlexNet

AlexNet was implemented with 35,855,178 parameters and trained for ten epochs. The final training history showed a steady improvement in training accuracy.

Table 3: AlexNet Training and Evaluation Results

Metric	Value
Total Parameters	35,855,178
Training Epochs	10
Best Validation Accuracy	70.20%
Test Accuracy	66.78%
Training Time	269.93 s

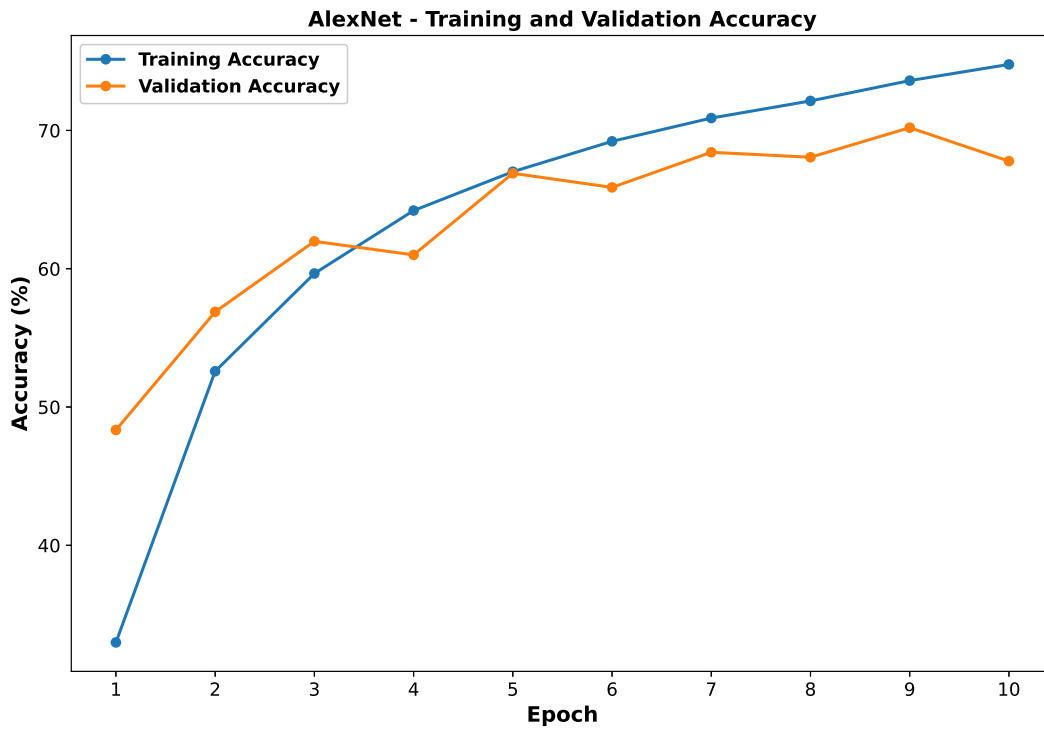


Figure 6: Training and validation accuracy of AlexNet

Observation:

AlexNet performed better than LeNet-5, with the best validation accuracy reaching 70.20%. Its training accuracy also increased from 32.98% in the first epoch to 74.77% by the tenth epoch. So, compared to LeNet-5, AlexNet was able to learn the image features better during training.

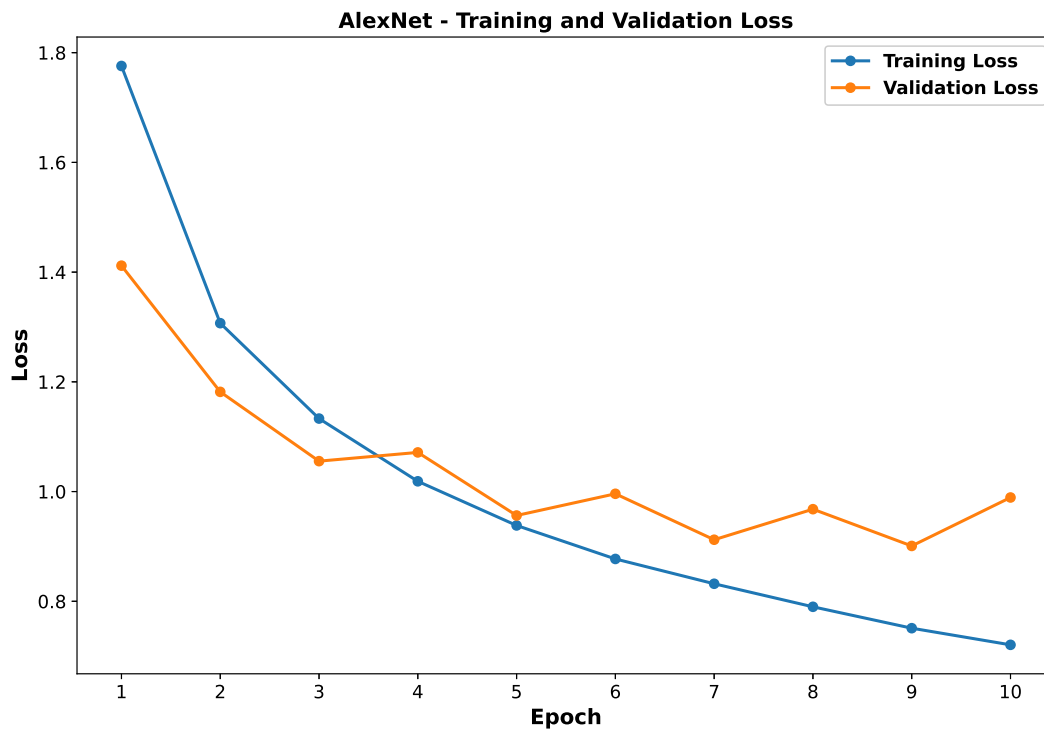


Figure 7: Training and validation loss of AlexNet

Observation:

The training loss kept coming down and reached 0.7206 by the last epoch, which shows that AlexNet was learning well from the training data. The validation loss had some ups and downs during the epochs,

so the improvement on the validation data was not as consistent as the training performance.

16.5 GoogleNet

The implemented GoogleNet model used Inception modules to extract features at different spatial scales. The model contained 223,150 parameters.

Table 4: GoogleNet Training and Evaluation Results

Metric	Value
Total Parameters	223,150
Training Epochs	10
Best Validation Accuracy	74.90%
Test Accuracy	73.37%
Training Time	133.26 s

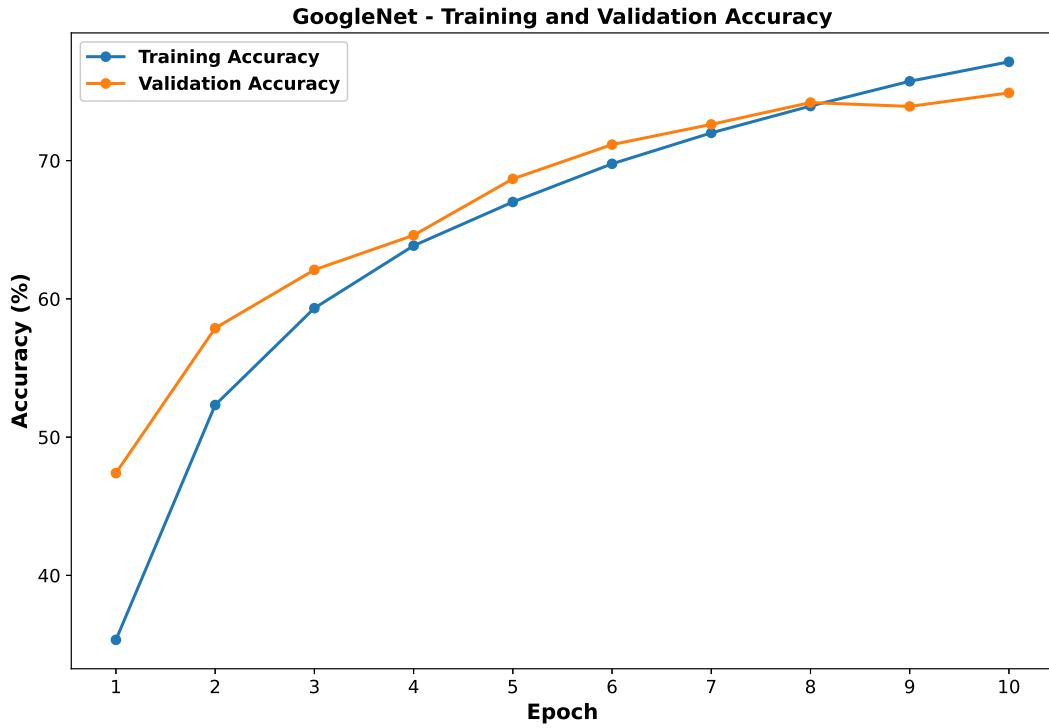


Figure 8: Training and validation accuracy of GoogleNet

Observation:

GoogleNet showed a steady improvement during training. The validation accuracy went from 47.40% in the first epoch to a best of 74.90%, while the training accuracy reached 77.15% by the tenth epoch. Compared to LeNet-5 and AlexNet, GoogleNet gave a better validation performance and was able to learn the CIFAR-10 features more effectively.

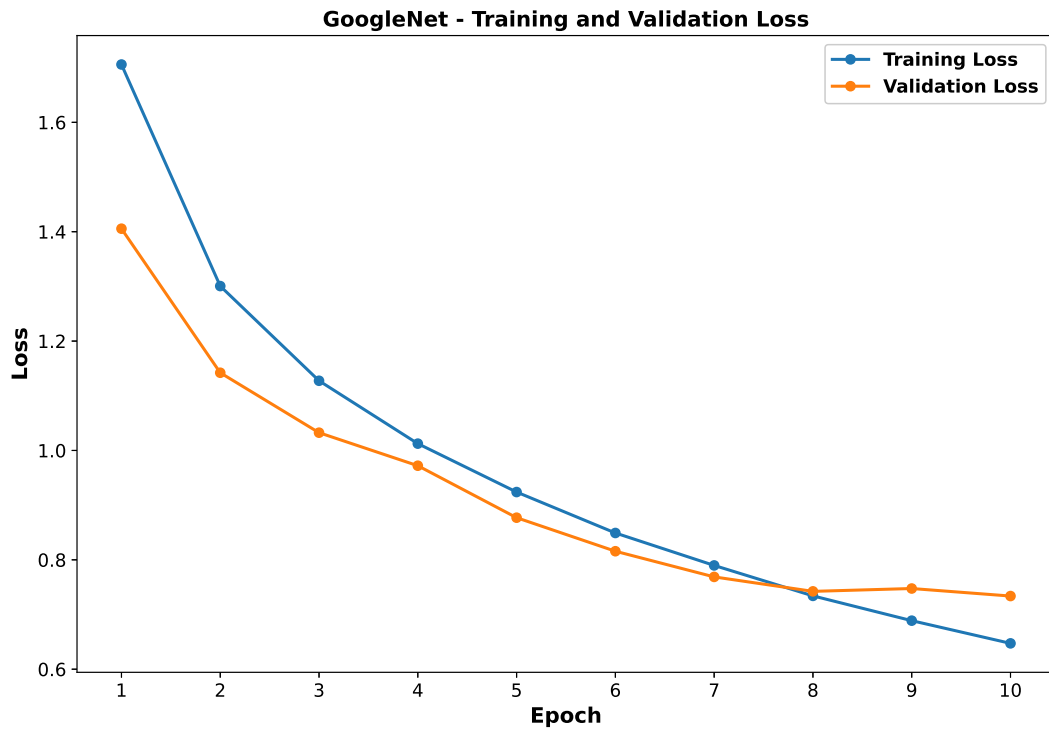


Figure 9: Training and validation loss of GoogleNet

Observation:

The training loss came down from 1.7057 in the first epoch to 0.6472 by the tenth epoch. The validation loss also decreased steadily and reached 0.7338 at the end. Overall, GoogleNet showed a fairly stable learning pattern, with both training and validation loss improving as the epochs progressed.

16.6 ResNet50

ResNet50 was the deepest model used in the comparison. The implemented model contained 21,299,146 parameters and was trained for ten epochs.

Table 5: ResNet50 Training and Evaluation Results

Metric	Value
Total Parameters	21,299,146
Training Epochs	10
Best Validation Accuracy	80.68%
Test Accuracy	79.77%
Training Time	1502.65 s

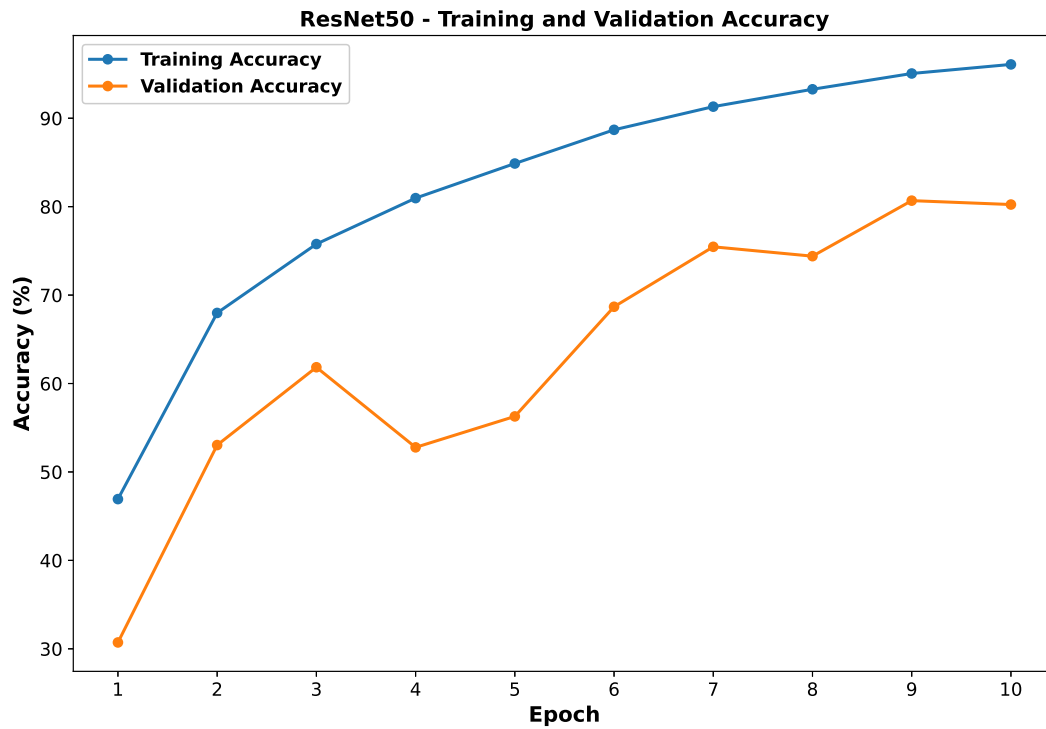


Figure 10: Training and validation accuracy of ResNet50

Observation:

ResNet50 showed the best training performance among the models. Its training accuracy increased from 46.93% in the first epoch to 96.08% by the tenth epoch, while the best validation accuracy was 80.68% in epoch 9. The difference between the two accuracies is quite noticeable, which shows that the model learned the training data very well but did not perform equally well on unseen images.

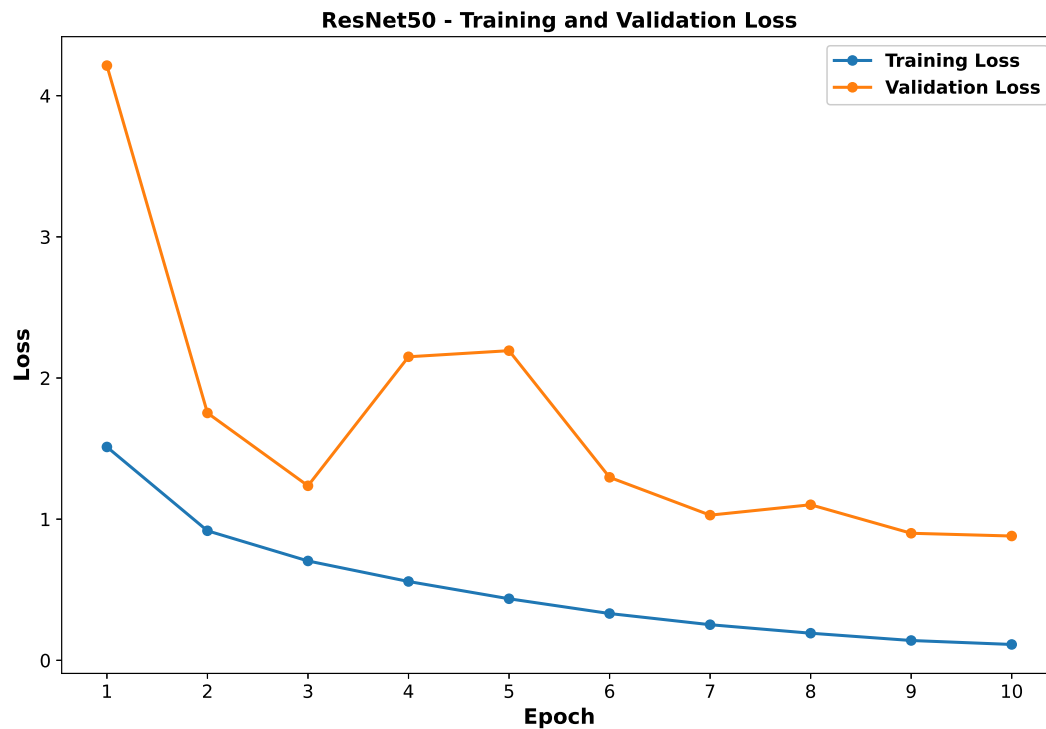


Figure 11: Training and validation loss of ResNet50

Observation:

The training loss dropped from 1.5115 in the first epoch to 0.1123 in the last epoch, showing that ResNet50 learned the training data quite well. The validation loss also improved overall, even though there were some fluctuations in the earlier epochs. On the test set, the model achieved a loss of 0.9062 and an accuracy of 79.77%, which was the best test accuracy among the models tested..

16.7 VGG16 Transfer Learning

VGG16 was used for the transfer learning part of the experiment. The ImageNet-pretrained convolutional base was first frozen and a new classification head was trained for the ten CIFAR-10 classes.

Table 6: VGG16 Frozen Base Results

Metric	Value
Total Parameters	14,781,642
Trainable Parameters	66,954
Best Validation Accuracy	65.12%
Final Validation Accuracy	63.58%
Training Time	136.08 s

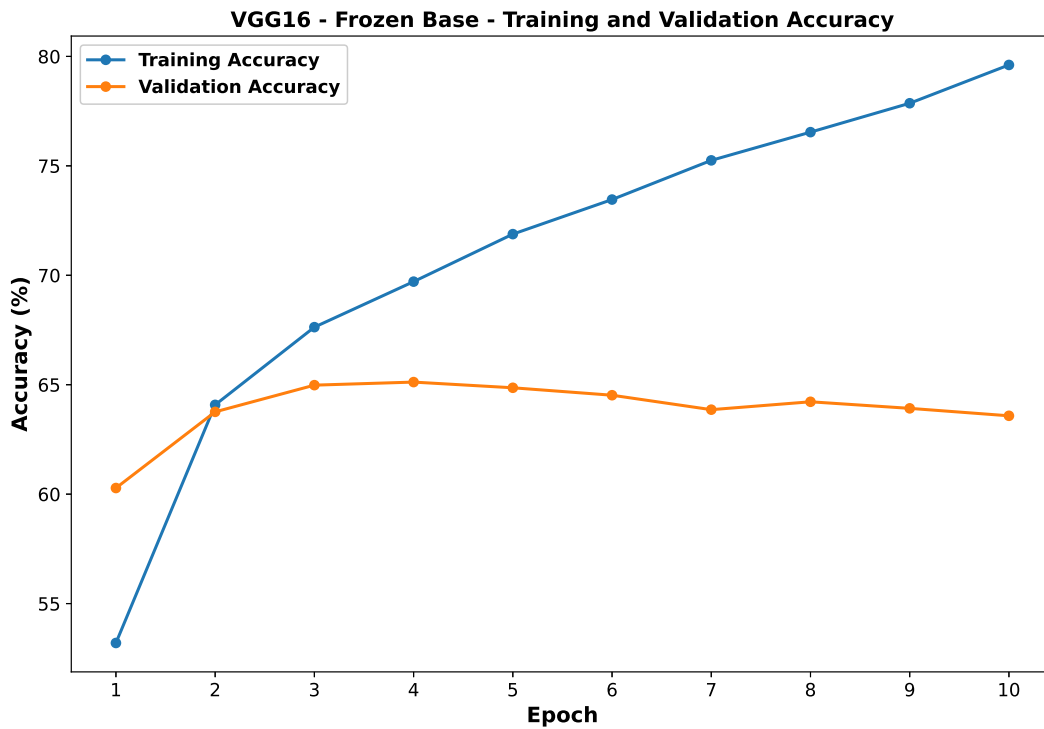


Figure 12: Frozen VGG16 training and validation accuracy

Observation:

The frozen VGG16 model was able to learn the new classification layer, but the validation accuracy was still limited to 65.12%. Since the pretrained layers were kept frozen, the features could not adjust much to the CIFAR-10 images. This is likely why the model did not perform as well as expected at this stage.

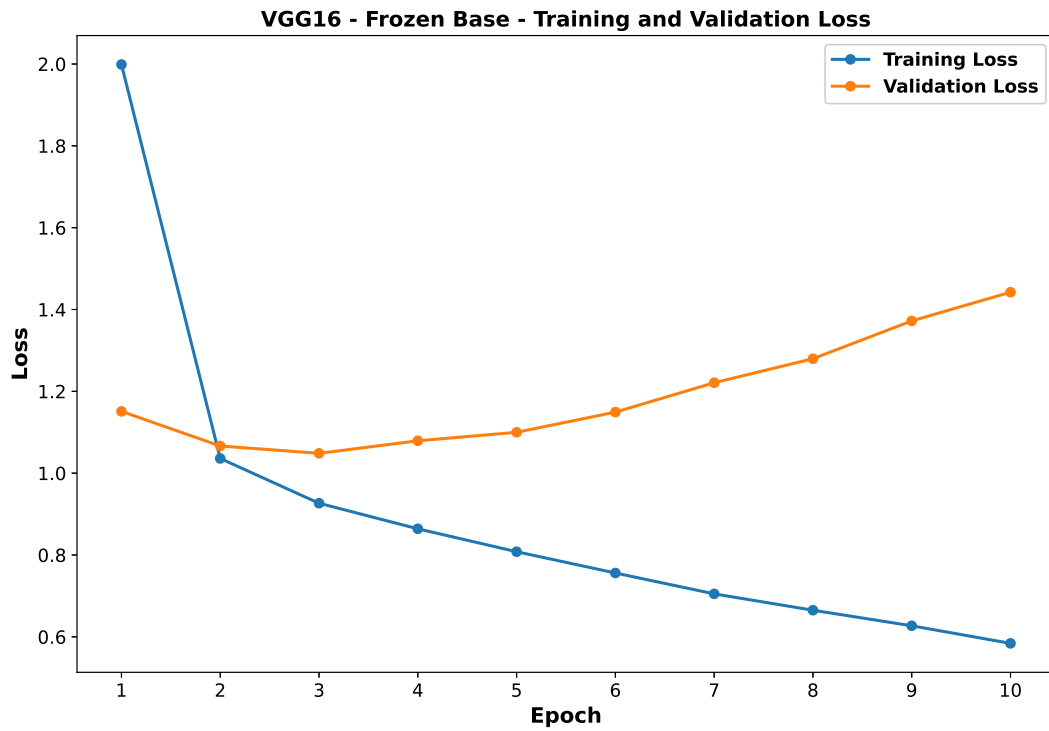


Figure 13: Frozen VGG16 training and validation loss

Observation:

The training loss kept decreasing, but the validation loss started going up in the later epochs. This means that the classifier was getting better on the training images, but the improvement was not carrying over to the validation images. Since the convolutional base was kept frozen, the model had limited scope to adjust its features to the CIFAR-10 data.

16.8 VGG16 Fine Tuning

After the frozen-base stage, the last convolutional block of VGG16 was unfrozen and trained for five additional epochs. The purpose was to allow the pretrained features to adapt to the CIFAR-10 dataset.

Table 7: VGG16 Frozen and Fine-Tuned Comparison

Metric	Frozen VGG16	Fine-Tuned VGG16
Best Validation Accuracy	65.12%	77.40%
Final Validation Accuracy	63.58%	77.16%
Training Time	136.08 s	89.17 s
Improvement	0.00%	12.28%

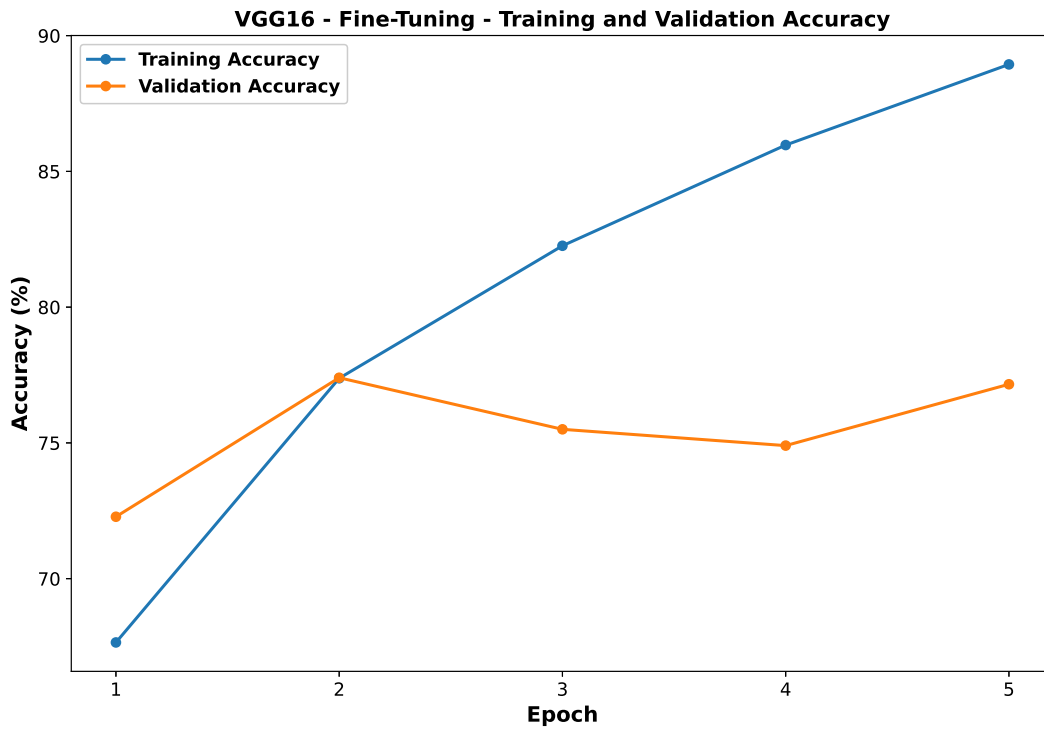


Figure 14: VGG16 fine-tuning accuracy

Observation:

Fine tuning made a noticeable difference in the validation performance. The best validation accuracy increased from 65.12% to 77.40%, which is an improvement of 12.28 percentage points. This shows that unfreezing part of VGG16 allowed the pretrained features to adjust better to the CIFAR-10 images.

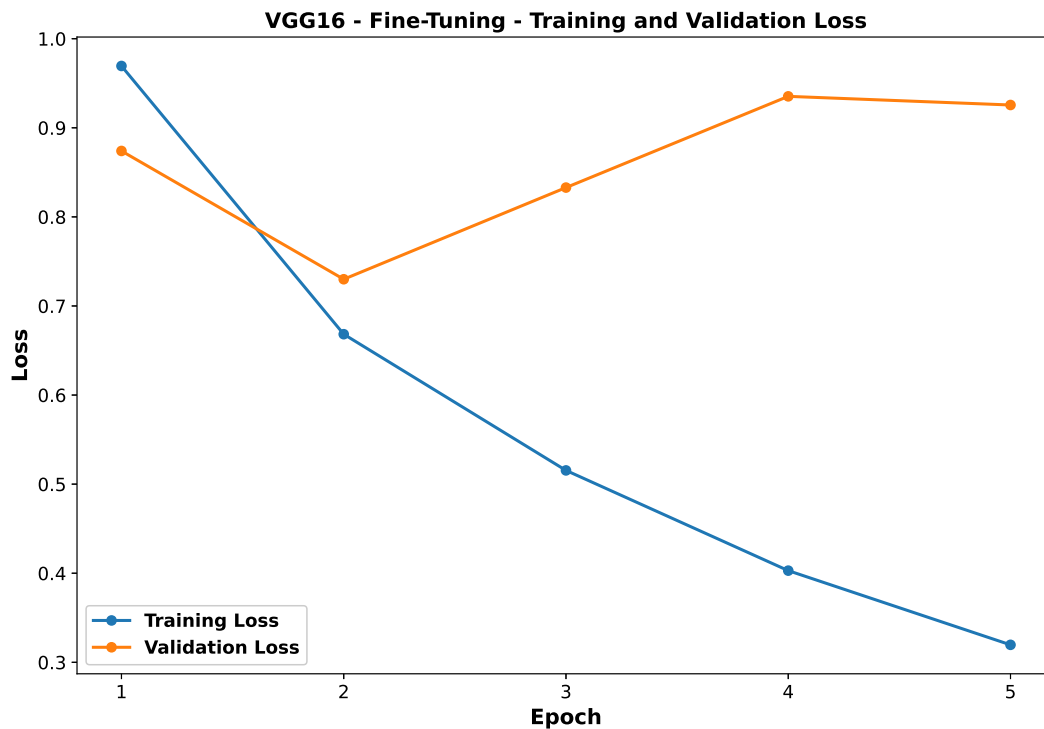


Figure 15: VGG16 fine-tuning loss

Observation:

During fine tuning, the training loss continued to decrease as the unfrozen layers adjusted to the CIFAR-10 images. Even though the validation loss had some small changes, the validation accuracy

improved clearly from 65.12% to 77.40%. So, fine tuning helped the model perform better on the validation data.

16.9 Hyperparameter Study

A separate hyperparameter study was performed using VGG16. The baseline configuration used Adam, a learning rate of 0.001, batch size 32, 10 epochs and 128 dense units with all convolutional layers frozen.

Table 8: Hyperparameter Study Results

Experiment	LR	Opt.	Batch	Epochs	Dense	Freeze	Val.	Test
Baseline	0.001	Adam	32	10	128	All	61.06	60.06
LR 0.0001	0.0001	Adam	32	10	128	All	59.86	57.91
Batch 16	0.001	Adam	16	10	128	All	60.92	59.21
Batch 64	0.001	Adam	64	10	128	All	61.54	60.09
Epochs 20	0.001	Adam	32	20	128	All	61.34	59.44
SGD	0.001	SGD	32	10	128	All	57.26	56.45
Dense 256	0.001	Adam	32	10	256	All	61.66	60.51
Partial Freeze	0.001	Adam	32	10	128	Partial	73.28	70.32

Observation:

Most of the changes like the learning rate, batch size, optimizer, number of epochs and dense units, did not make a big difference in the test accuracy. The main improvement came from partial freezing, where the test accuracy increased from 60.06% for the baseline to 70.32%. This shows that allowing some of the pretrained layers to adapt was more useful than changing the other parameters individually.

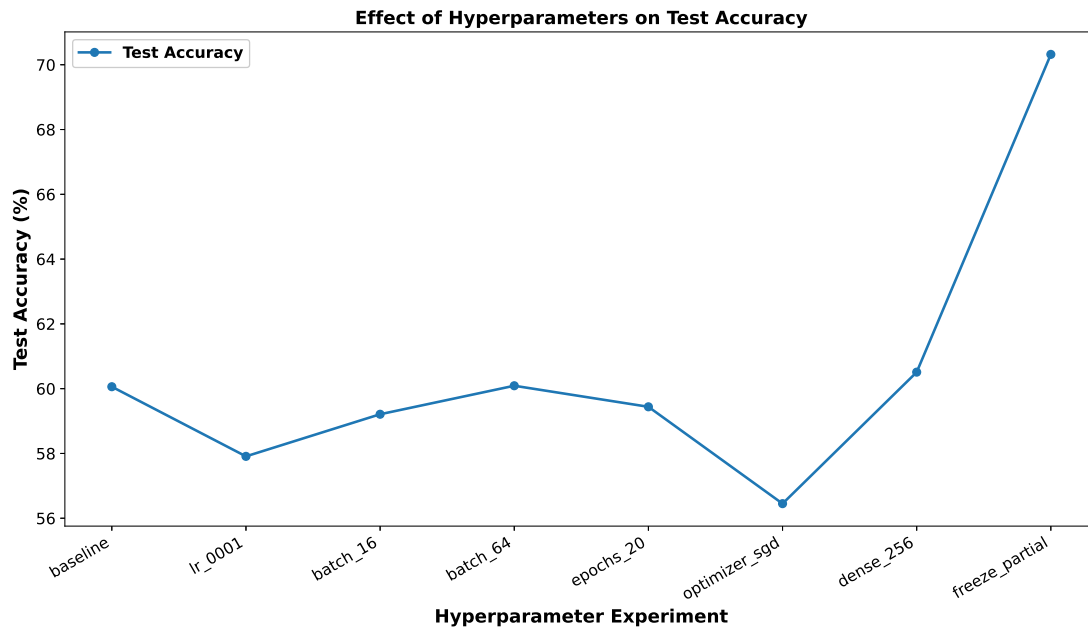


Figure 16: Test accuracy for the different hyperparameter configurations

Observation:

The partial-freezing setup performed noticeably better than all the other configurations tested. The other experiments stayed between 56.45% and 60.51% test accuracy, while partial freezing reached 70.32%. This makes partial freezing the most effective setting among the hyperparameter configurations tested.

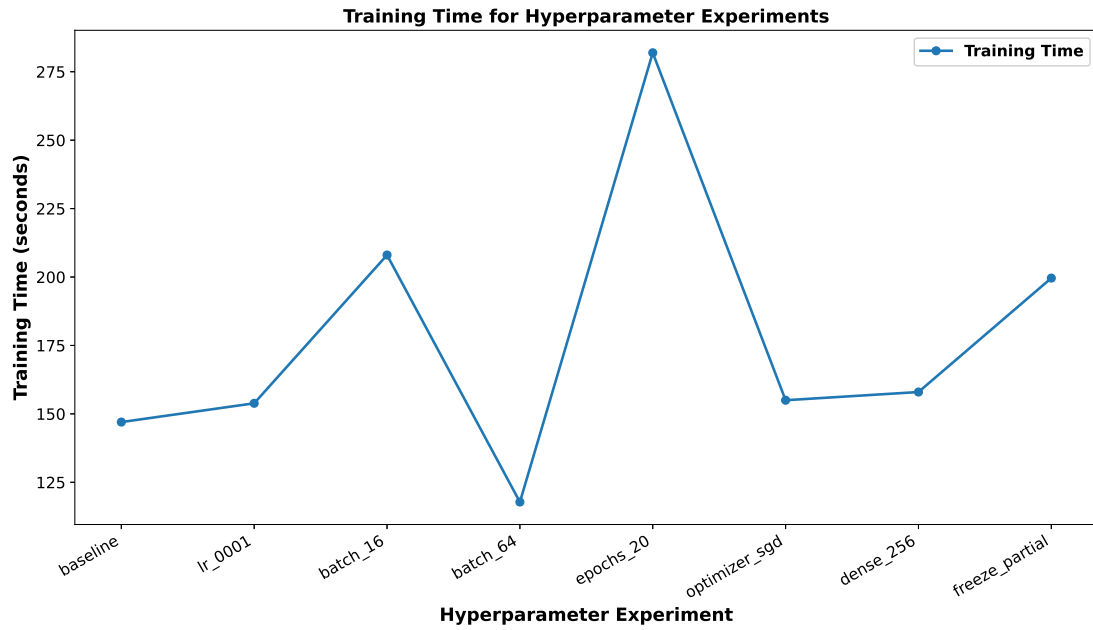


Figure 17: Training time for the different hyperparameter configurations

Observation:

Using a batch size of 64 gave the shortest training time of 117.78 seconds. The experiment with 20 epochs took the longest at 281.93 seconds. However, increasing the number of epochs did not improve the test accuracy much, so in this case it mainly increased the training time without giving a useful performance improvement.

16.10 Final Classification Metrics

The final models were evaluated on the 10,000-image CIFAR-10 test set using accuracy, precision, recall and F1-score.

Table 9: Final Classification Metrics

Model	Accuracy	Precision	Recall	F1-score
LeNet-5	62.04%	62.11%	62.04%	61.50%
AlexNet	66.78%	68.09%	66.78%	66.45%
GoogleNet	73.37%	73.55%	73.37%	73.07%
VGG16	76.90%	77.95%	76.90%	77.13%
ResNet50	79.77%	80.49%	79.77%	79.74%

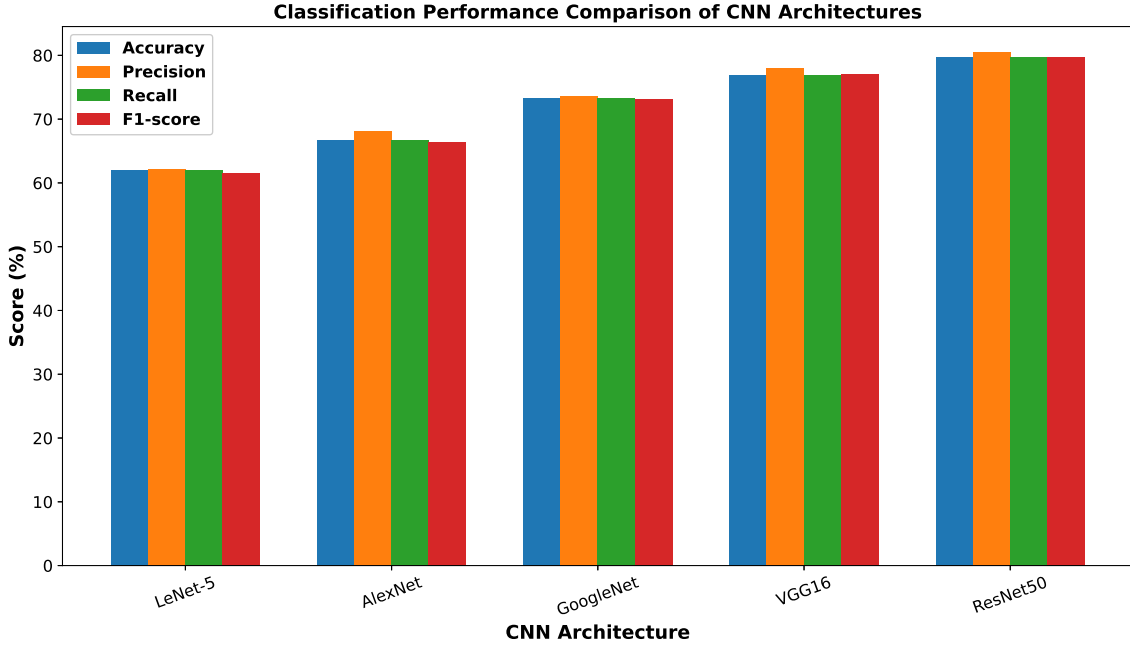


Figure 18: Comparison of final classification metrics

Observation:

ResNet50 performed the best across all four metrics, while LeNet-5 had the lowest scores overall. GoogleNet and VGG16 were in between the two. The precision, recall and F1-score of ResNet50 were also very close to each other, which shows that the model gave fairly consistent results across the test set.

16.11 Class-wise Evaluation

The class-wise classification report was also examined to understand which CIFAR-10 classes were easier and harder for the models to classify.

Table 10: Class-wise Performance of ResNet50

Class	Precision	Recall	F1-score
Airplane	83.45%	83.20%	83.32%
Automobile	93.09%	86.20%	89.51%
Bird	74.79%	72.10%	73.42%
Cat	70.00%	55.30%	61.79%
Deer	80.18%	73.20%	76.53%
Dog	62.02%	84.10%	71.39%
Frog	89.16%	77.30%	82.81%
Horse	75.55%	89.60%	81.98%
Ship	86.96%	88.70%	87.82%
Truck	89.70%	88.00%	88.84%

Observation:

Among the ResNet50 classes, automobile had the highest precision at 93.09%, while horse had the highest recall at 89.60%. On the other hand, cat was the hardest class to classify, with a recall of only 55.30%. This could be because some of the CIFAR-10 images are low in resolution and a few classes have similar visual features, making them harder to distinguish.

16.12 Confusion Matrix Analysis

The confusion matrices were generated for the individual CNN models to observe the class-wise prediction behaviour.

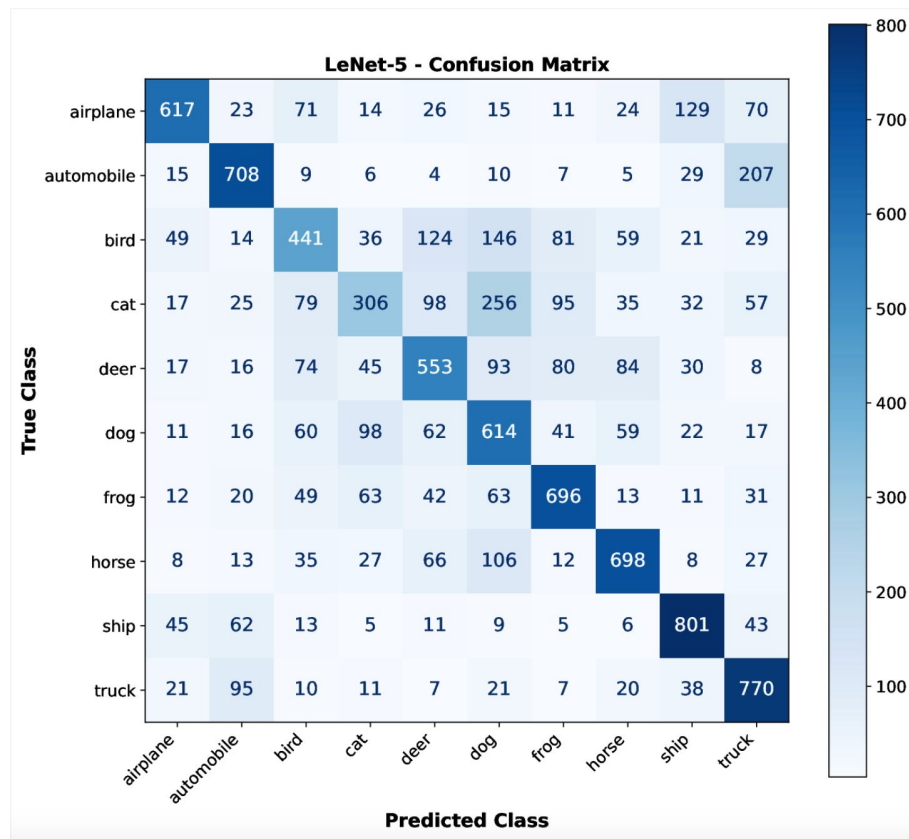


Figure 19: Confusion matrix of LeNet-5

Observation:

LeNet-5 performed relatively well on classes such as ship, frog and horse. Cat was particularly difficult, with a recall of only 30.60%, indicating that many cat images were assigned to other classes.

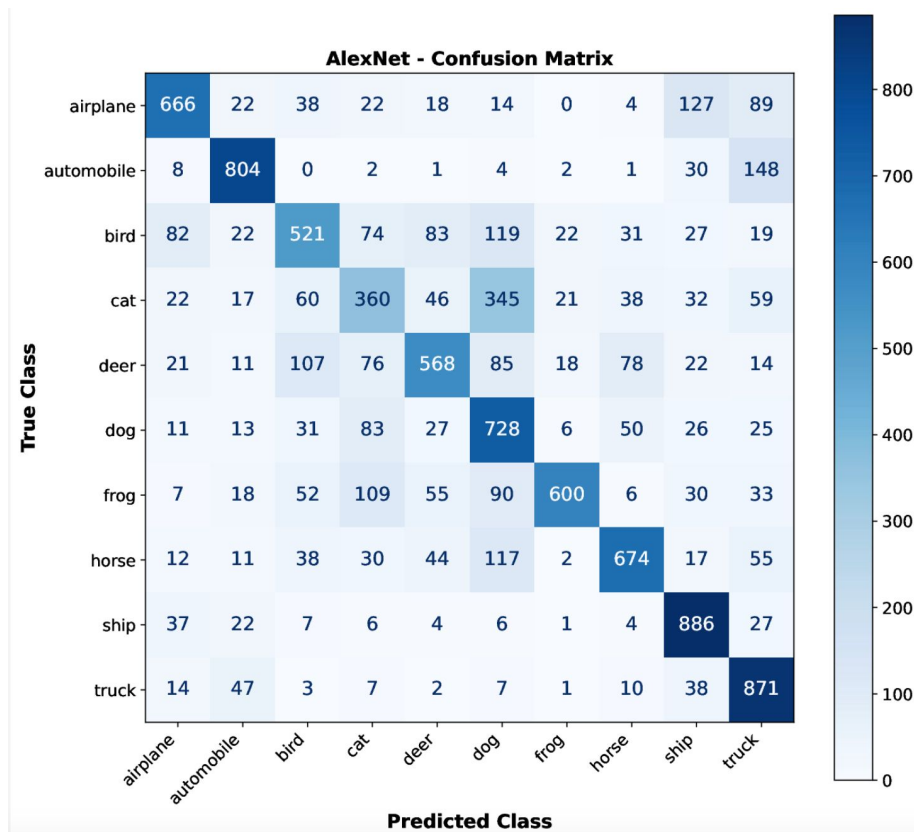


Figure 20: Confusion matrix of AlexNet

Observation:

AlexNet did better than LeNet-5 for several classes. Ship and truck had the highest recall values of 88.60% and 87.10%, respectively. However, cat was still difficult for the model, with a recall of only 36.00

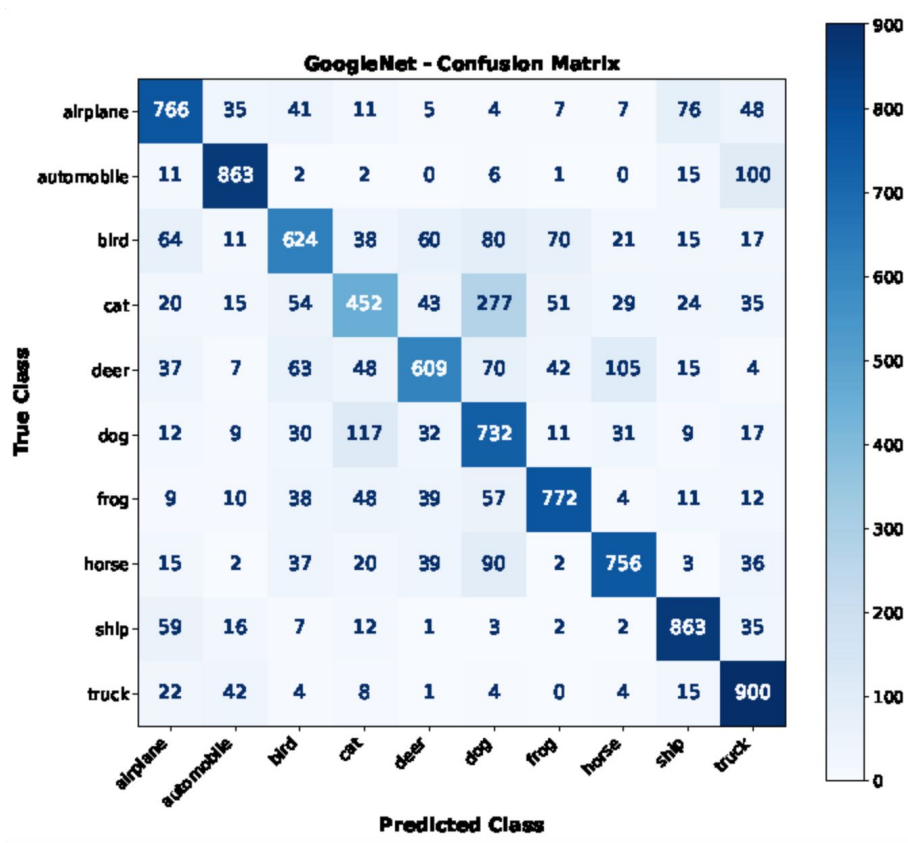


Figure 21: Confusion matrix of GoogleNet

Observation:

GoogleNet showed better class-wise performance compared to LeNet-5 and AlexNet. Truck had the highest recall at 90.00%, while cat was still one of the harder classes for the model, with a recall of 45.20

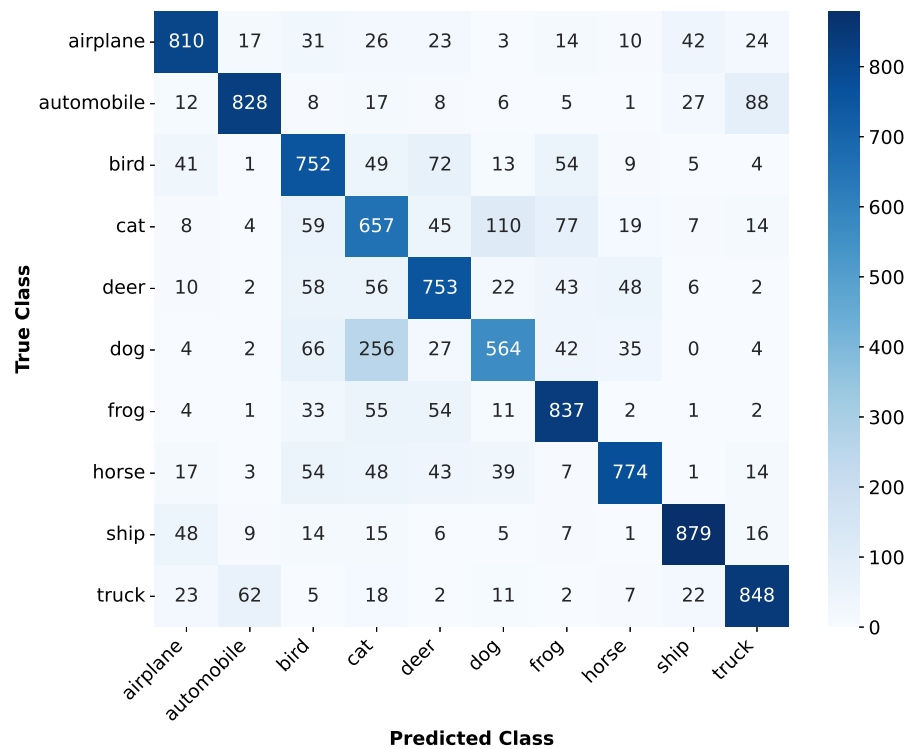


Figure 22: Confusion matrix of VGG16

Observation:

VGG16 gave a more balanced performance across the different classes compared to the smaller models. Ship had the highest recall at 83.30%, while cat was still comparatively difficult, with a recall of 69.1%.

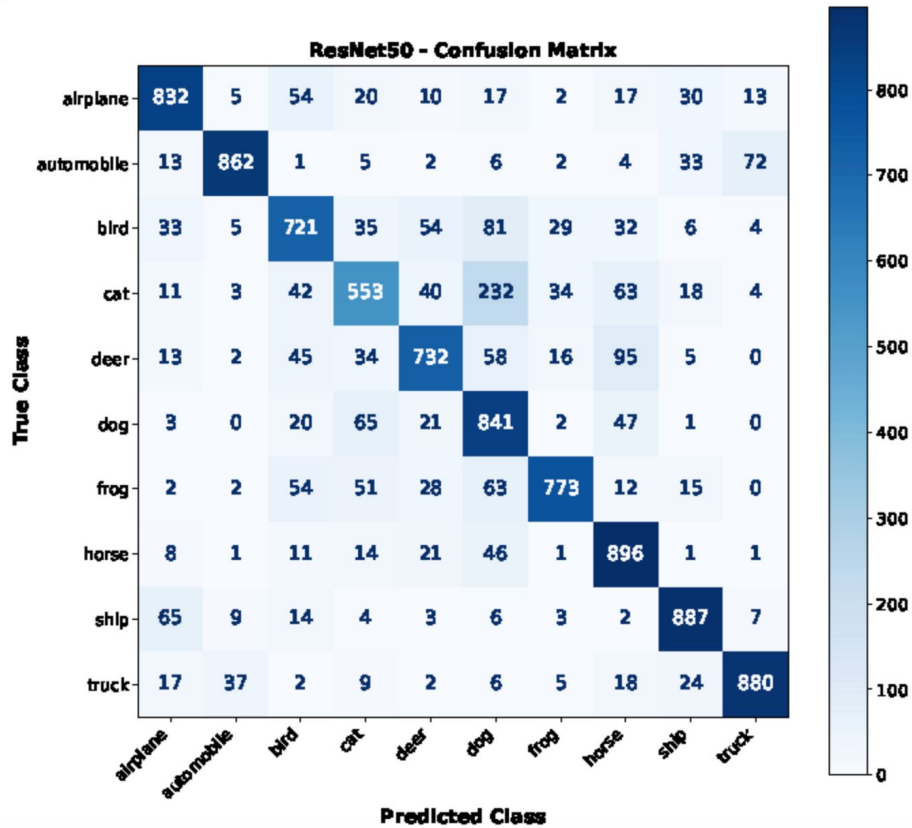


Figure 23: Confusion matrix of ResNet50

Observation:

ResNet50 gave the best class-wise performance among the models. Horse, ship, and truck had high recall values of 89.60%, 88.70%, and 88.00%, respectively. However, cat was still the weakest class, with a recall of 55.30%, showing that this class was harder for the model to identify correctly.

17. Mandatory Plots

The following plots were included as required by the laboratory manual. Each plot is accompanied by a brief observation based on the actual experimental results.

17.1 Sample CIFAR-10 Images

The sample images used during dataset preparation are shown in Figure 3.

Inference:

The images show the variety of objects present in the CIFAR-10 dataset. The low resolution of the images makes fine visual details less distinct, which contributes to confusion between visually similar classes.

17.2 Training Accuracy

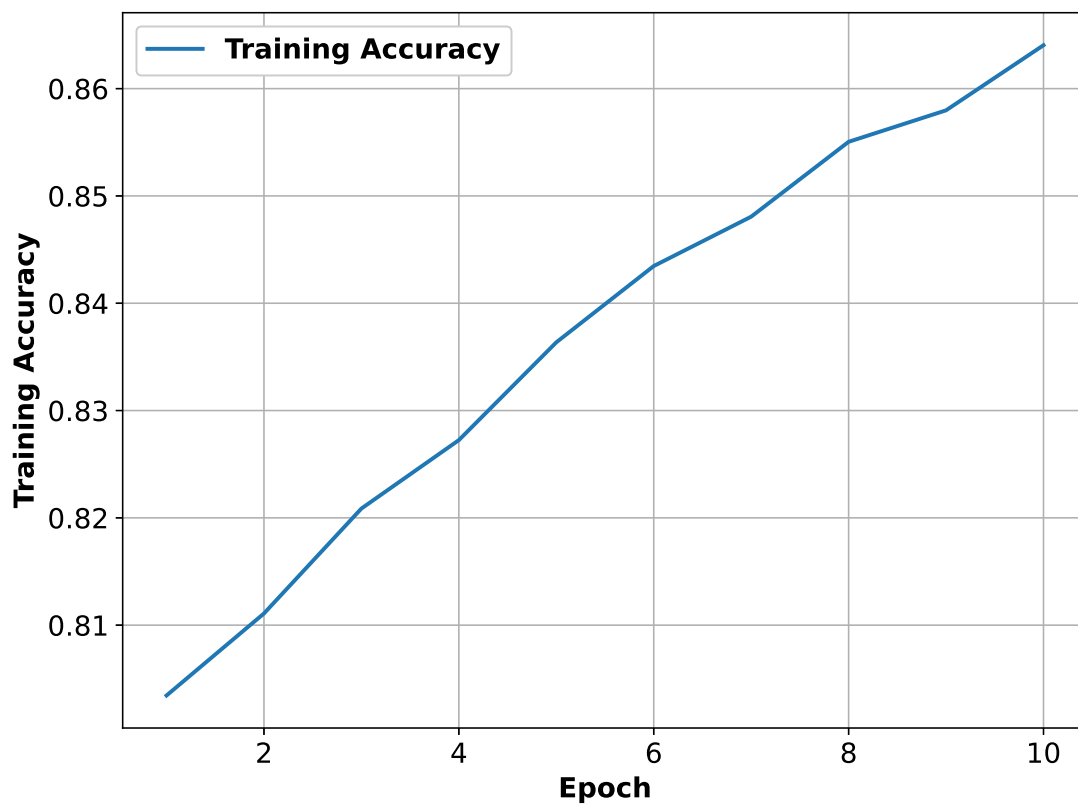


Figure 24: Training accuracy

Inference:

The training accuracy increased over the epochs for all the models, showing that they were gradually learning from the training data. Among them, ResNet50 showed the biggest improvement, reaching 96.08% training accuracy by the end of the 10 epochs.

17.3 Validation Accuracy

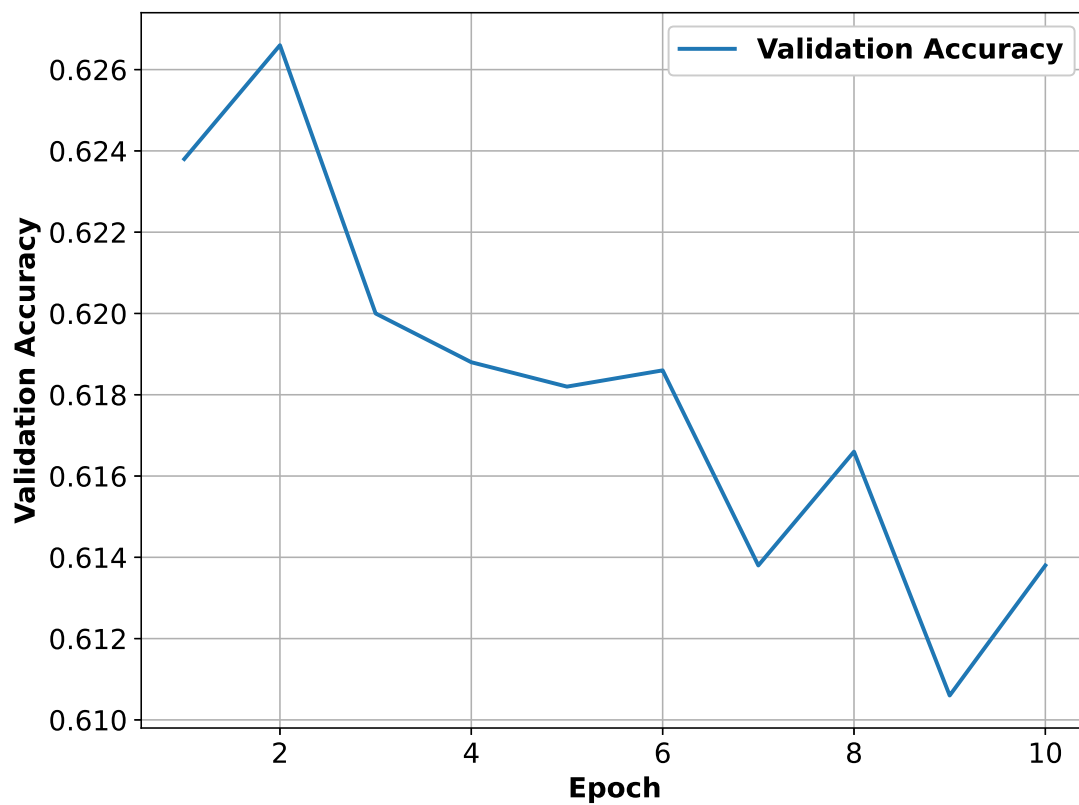


Figure 25: Validation accuracy

Inference:

The validation accuracy generally improved as training progressed, but it did not always increase along with the training accuracy. This was more noticeable in the deeper models, where the training accuracy became much higher than the validation accuracy. This shows that the models were learning the training images very well, but the improvement was not always reflected on unseen images.

17.4 Training Loss

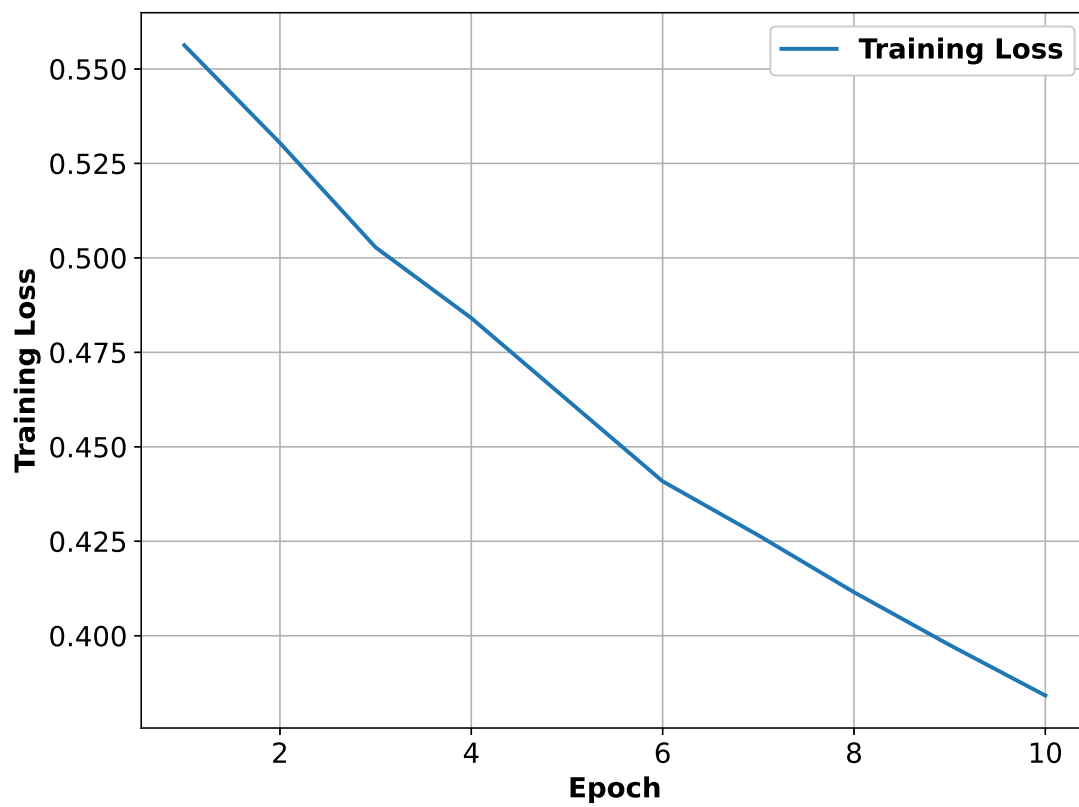


Figure 26: Training loss

Inference:

The training loss kept decreasing as the models learned from the training data. This decrease was especially clear in ResNet50, where the loss dropped from 1.5115 in the first epoch to just 0.1123 by the tenth epoch.

17.5 Validation Loss

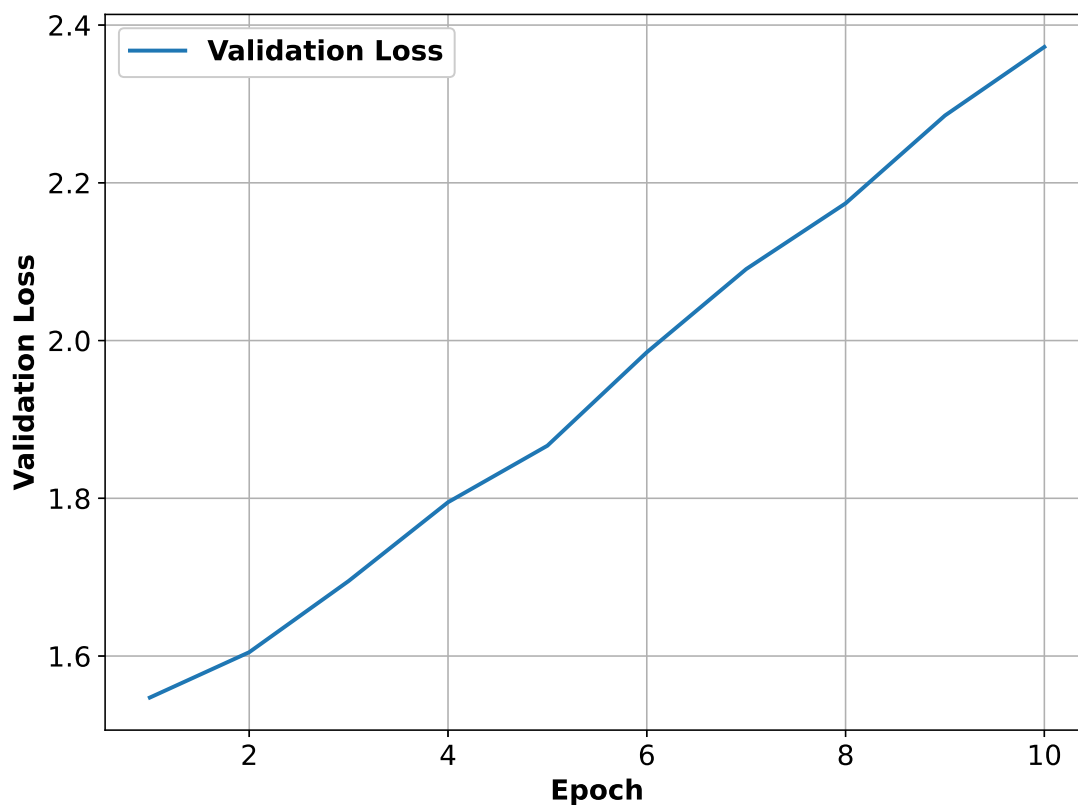


Figure 27: Validation loss

Inference:

The validation loss generally decreased during the initial stages of training, but it started to fluctuate for some of the models in the later epochs. So, even though the training loss kept going down, it did not always mean that the model was improving by the same amount on the validation data.

17.6 Confusion Matrix

The final ResNet50 confusion matrix is used as the main confusion matrix for the mandatory plot because ResNet50 obtained the highest test accuracy.

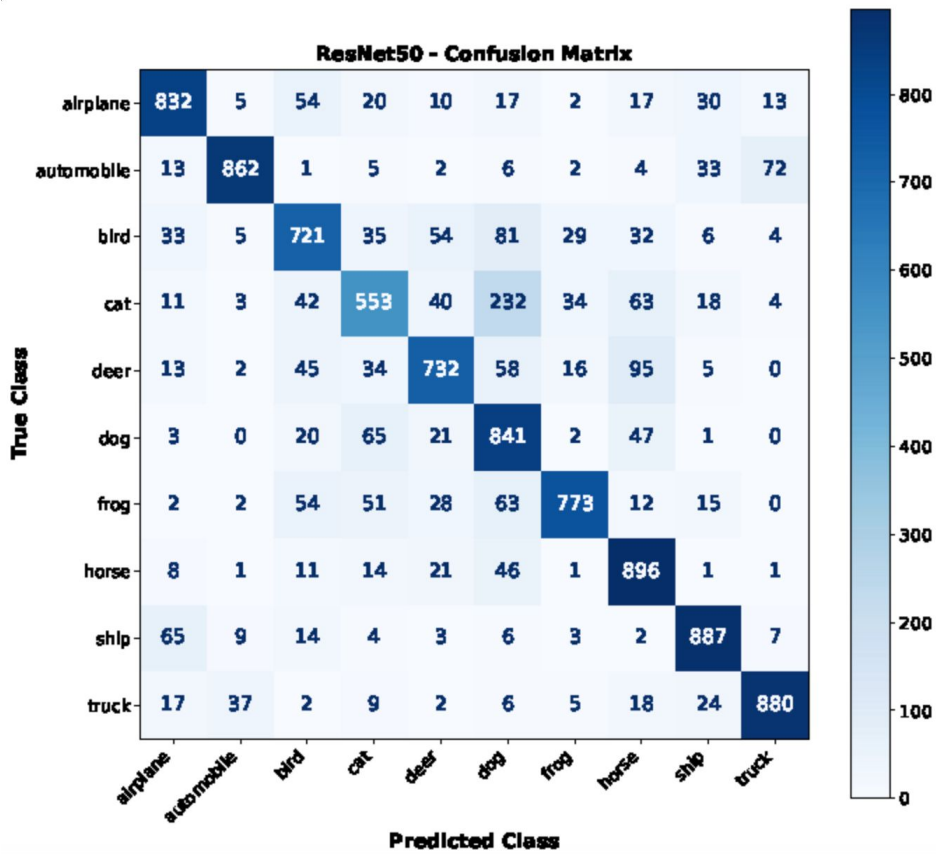


Figure 28: Final confusion matrix of ResNet50

Inference:

The confusion matrix shows that most of the predictions are along the diagonal, which means the model classified a good number of test images correctly. Cat was still one of the more difficult classes, while horse, ship, and truck were classified more consistently.

17.7 Misclassified Images

The misclassified images give a better idea of the cases where the model was not able to identify the correct class. Some of these errors are expected because the CIFAR-10 images are small and a few classes have similar visual features. Looking at these images also helps explain why classes such as cat were harder to classify compared to classes such as ship, horse and truck.

18. Results

The final results were taken from the test set and compared using accuracy, precision, recall, F1-score, parameter count, and training time. This makes it easier to see not only which model gave the best accuracy, but also how much computation each model required.

18.1 Performance Metrics

ResNet50 was the best-performing architecture based on the final test accuracy. Its main performance metrics are shown below.

Table 11: Performance Metrics of the Best-Performing Model

Metric	Value
Training Accuracy	96.08%
Testing Accuracy	79.77%
Precision	80.49%
Recall	79.77%
F1-score	79.74%
Training Time	1502.65 s
Total Parameters	21,299,146

Interpretation:

ResNet50 gave the highest test accuracy of 79.77% among the models. Its precision, recall and F1-score were also close to this value, which shows that the model performed fairly consistently across the test set. The training accuracy was much higher at 96.08%, so there is a noticeable gap between training and test performance, indicating some overfitting.

18.2 Comparison of CNN Architectures

Table 12: Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	136,886	62.04	60.56
AlexNet	35,855,178	66.78	269.93
VGG16	14,781,642	76.90	225.25
GoogleNet	223,150	73.37	133.26
ResNet50	21,299,146	79.77	1502.65

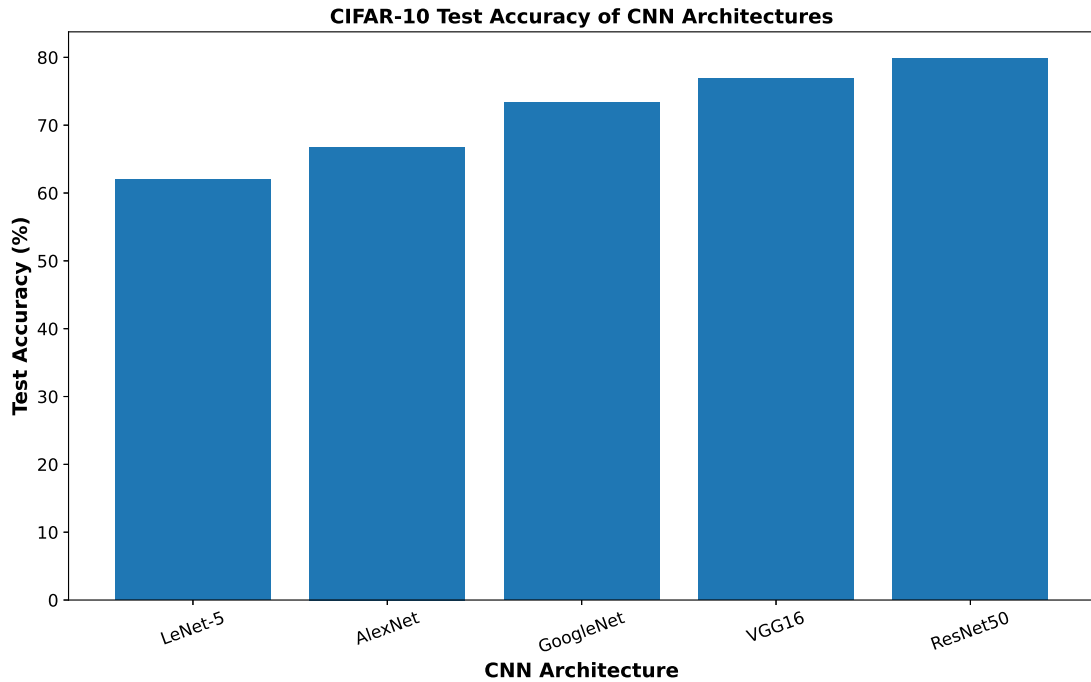


Figure 29: Comparison of test accuracy of CNN architectures

Interpretation:

ResNet50 gave the highest test accuracy at 79.77%, followed by VGG16 with 76.90% and GoogleNet with 73.37%. LeNet-5 had the lowest accuracy at 62.04%, but it was also the quickest model to train. So, while the deeper models gave better accuracy, they also required more time and computation.

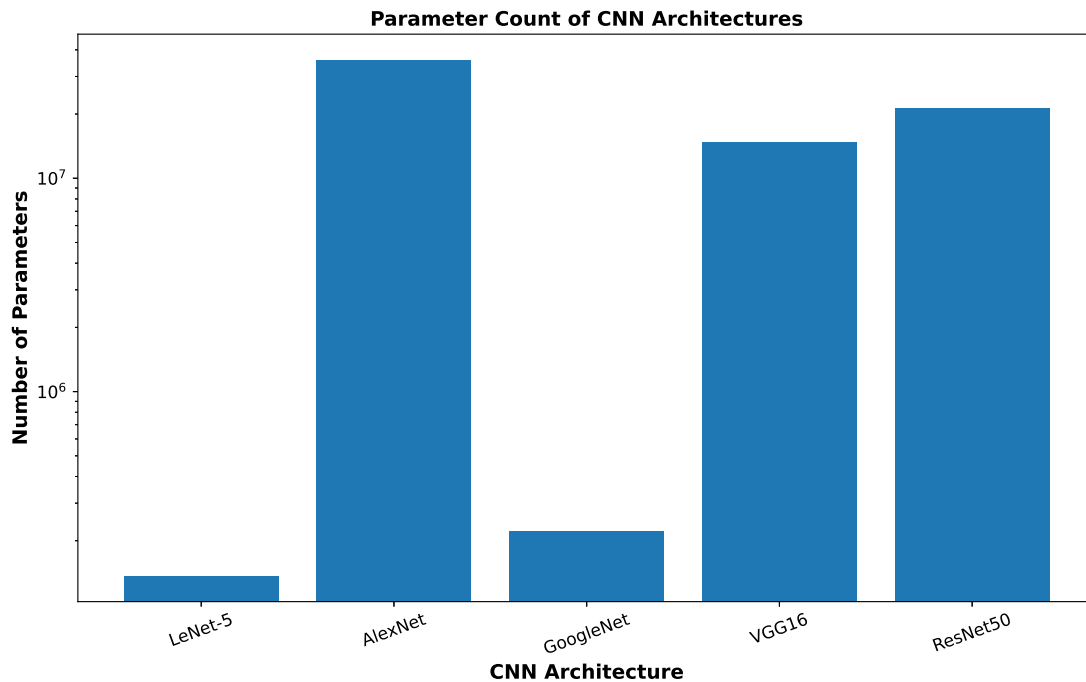


Figure 30: Parameter count of the CNN architectures

Interpretation:

The number of parameters did not directly decide how well the models performed. For example, GoogleNet had only 223,150 parameters but still achieved 73.37% test accuracy, which was better than AlexNet's 66.78%, even though AlexNet had more than 35 million parameters. This shows that the way the network is designed also has a major effect on the final performance.

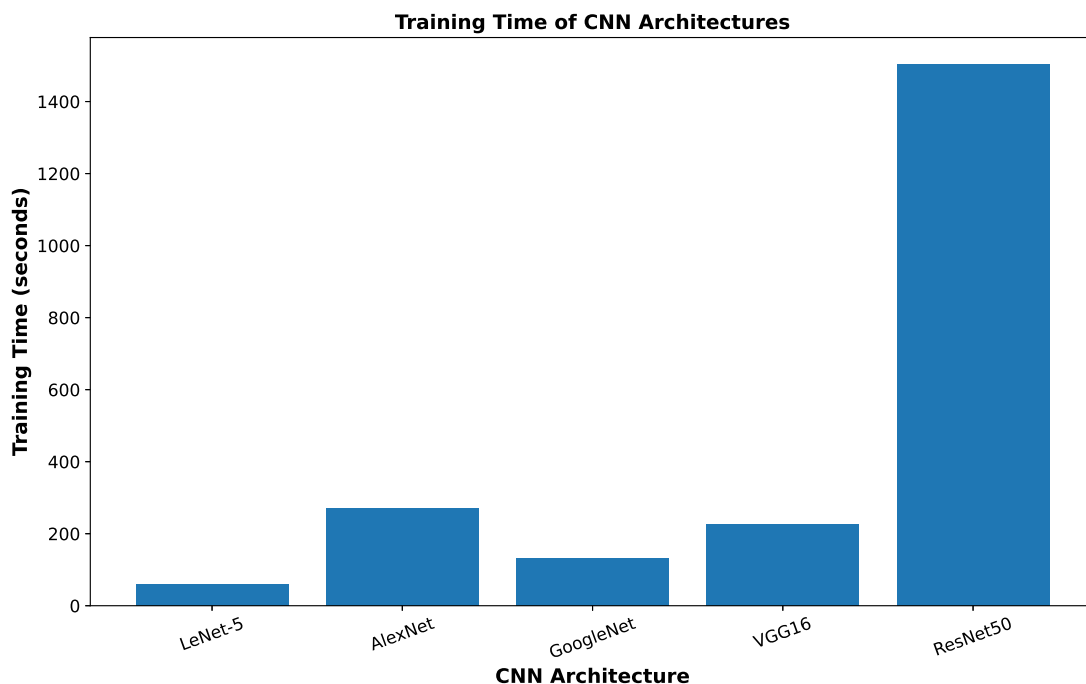


Figure 31: Training time of the CNN architectures

Interpretation:

ResNet50 took much longer to train compared to the other models, with a training time of 1502.65 seconds, while LeNet-5 took only 60.56 seconds. So, although the deeper model gave better accuracy, it also required much more computation and training time.

19. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet was one of the important steps in the development of deep CNNs because it showed how a deeper network could perform well on image classification when trained with a large dataset and GPU support. It also used ReLU and Dropout effectively. In our experiment, AlexNet achieved 66.78% test accuracy, which was better than the 62.04% obtained by LeNet-5.

2. Why does VGG16 use only 3×3 convolution filters?

VGG16 uses multiple 3×3 convolution filters instead of using larger filters. By stacking these smaller filters, the network can learn more detailed features as the layers go deeper. In our experiment, VGG16 achieved a test accuracy of 76.90%, which was one of the better results among the models tested.

3. Explain the advantages of the Inception module.

The Inception module uses different convolution operations in parallel and then combines their outputs. This helps GoogleNet capture features at different sizes without making the network too large. In our experiment, GoogleNet achieved 73.37% test accuracy with only 223,150 parameters, so it gave a good balance between model size and classification performance.

4. What is the purpose of residual learning?

Residual learning allows a deep network to learn the difference, or residual, instead of trying to learn the complete mapping directly. The skip connections also make it easier for information to pass through the deeper layers. In our experiment, this helped ResNet50 achieve the highest test accuracy of 79.77% among all the models tested.

5. Differentiate LeNet and ResNet.

LeNet-5 is a simpler and smaller CNN, with basic convolution, pooling and fully connected layers, while ResNet50 is much deeper and uses skip connections to handle the deeper network. In our experiment, LeNet-5 had 136,886 parameters and took only 60.56 seconds to train, whereas ResNet50 had 21,299,146 parameters and took 1502.65 seconds. Even though ResNet50 required much more time and computation, it achieved a much higher test accuracy of 79.77% compared to 62.04% for LeNet-5.

6. What is Transfer Learning?

Transfer learning means using the features learned by a model on one dataset and adapting them to a different dataset. Instead of training the whole network from the beginning, the already learned features can be reused and adjusted for the new task. In our experiment, we used VGG16 pretrained on ImageNet and adapted it for the 10-class CIFAR-10 classification task.

7. Why is fine tuning required?

The pretrained features may not be a perfect fit for the new dataset, so fine tuning allows some of the VGG16 layers to adjust to the CIFAR-10 images. In our experiment, the best validation accuracy increased from 65.12% with the frozen base to 77.40% after fine tuning. This is an improvement of 12.28 percentage points, showing that fine tuning helped VGG16 adapt better to the CIFAR-10 classification task.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution uses gaps between the kernel elements, which helps the model look at a larger area of the image without greatly increasing the number of parameters. Transpose convolution, on the other hand, is mainly used to increase the spatial size of feature maps through learnable upsampling. So, the main difference is that dilated convolution increases the receptive field, while transpose convolution increases spatial resolution.

9. Why do pretrained models converge faster?

A pretrained model already has weights that have learned basic visual features from a large dataset. Because of this, it does not have to learn things like edges and textures from the beginning. In our experiment, the ImageNet-pretrained VGG16 provided a useful starting point, and the classifier was then adapted to the CIFAR-10 classes.

10. Compare the computational complexity of LeNet and ResNet.

The difference in computational cost was quite noticeable in the experiment. LeNet-5 had only 136,886 parameters and took 60.56 seconds to train, whereas ResNet50 had 21,299,146 parameters and took 1502.65 seconds. Even though ResNet50 required much more time and computation, it achieved a much higher test accuracy of 79.77% compared to 62.04% for LeNet-5. So, LeNet-5 would be a better choice when faster training and lower computation are more important, while ResNet50 is more suitable when better accuracy is the main priority.

20. Additional Exercises

1. Implement Transfer Learning Using VGG16

VGG16 was started with the pretrained ImageNet weights, and the original classifier was replaced with a new one for the 10 CIFAR-10 classes. First, the convolutional base was kept frozen and the new classifier was trained. After that, the last convolutional block was unfrozen so that it could be fine-tuned for the CIFAR-10 images.

Table 13: VGG16 Transfer Learning and Fine-Tuning

Metric	Frozen	Fine-Tuned
Best Validation Accuracy	65.12%	77.40%
Final Validation Accuracy	63.58%	77.16%
Training Time	136.08 s	89.17 s
Improvement	—	12.28 percentage points

Fine tuning gave a clear improvement in the validation results. The best validation accuracy increased from 65.12% to 77.40%, showing that allowing the pretrained layers to adapt helped VGG16 perform better on the CIFAR-10 images.

The training times in this experiment were recorded separately from the final CNN architecture comparison. The frozen VGG16 stage took 136.08 seconds and the fine-tuning stage took 89.17 seconds, whereas the VGG16 model used in the final architecture comparison recorded a training time of 225.25 seconds.

2. Repeat the Experiment Using ResNet50

ResNet50 was trained for ten epochs and evaluated on the CIFAR-10 test set.

Table 14: ResNet50 Results

Metric	Value
Best Validation Accuracy	80.68%
Test Accuracy	79.77%
Precision	80.49%
Recall	79.77%
F1-score	79.74%
Training Time	1502.65 s
Parameters	21,299,146

ResNet50 achieved the highest test accuracy of 79.77% among all the models tested. The main drawback was its much longer training time of 1502.65 seconds, so the better accuracy came with a higher computational cost.

3. Compare Adam and SGD Optimizers

The effect of the optimizer was studied while keeping the remaining baseline settings unchanged.

Table 15: Comparison of Adam and SGD

Metric	Adam	SGD
Learning Rate	0.001	0.001
Batch Size	32	32
Epochs	10	10
Best Validation Accuracy	61.06%	57.26%
Test Accuracy	60.06%	56.45%
Training Time	147.00 s	154.98 s

For the settings tested, Adam performed better than SGD. The test accuracy was 60.06% with Adam, compared to 56.45% with SGD, and the validation accuracy was also higher with Adam. So, Adam gave better results for the configuration used in this experiment.

4. Compare Training with Frozen Layers and Fine Tuning

The effect of freezing and fine tuning was examined using VGG16.

Table 16: Frozen Layers versus Fine Tuning

Metric	Frozen	Fine-Tuned
Best Validation Accuracy	65.12%	77.40%
Final Validation Accuracy	63.58%	77.16%
Training Time	136.08 s	89.17 s

Fine tuning improved the validation accuracy quite a lot. In our results, allowing some of the pretrained layers to be updated worked better than keeping the whole base frozen. This shows that partially unfreezing the pretrained layers helped the model adjust better to the CIFAR-10 images.

5. Evaluate the Model Using Precision, Recall and F1-score

The models were evaluated using multiple classification metrics so that the comparison was not based only on accuracy.

Table 17: Classification Metrics of the CNN Models

Model	Accuracy	Precision	Recall	F1-score
LeNet-5	62.04%	62.11%	62.04%	61.50%
AlexNet	66.78%	68.09%	66.78%	66.45%
GoogleNet	73.37%	73.55%	73.37%	73.07%
VGG16	76.90%	77.95%	76.90%	77.13%
ResNet50	79.77%	80.49%	79.77%	79.74%

ResNet50 gave the best results across all four metrics. Its precision, recall and F1-score were also quite close to its 79.77% accuracy, which shows that the model performed fairly consistently across the test set.

6. Compare LeNet, AlexNet and ResNet

Table 18: Comparison of LeNet-5, AlexNet and ResNet50

Model	Parameters	Test Accuracy	Training Time
LeNet-5	136,886	62.04%	60.56 s
AlexNet	35,855,178	66.78%	269.93 s
ResNet50	21,299,146	79.77%	1502.65 s

ResNet50 gave the best classification performance among the three models, but it also took much longer to train. LeNet-5 was the fastest, although its accuracy was lower. AlexNet gave a better accuracy than LeNet-5, but this came with a much larger number of parameters.

21. Conclusion

This experiment gave a practical comparison of different CNN architectures for image classification using the CIFAR-10 dataset. LeNet-5, AlexNet, GoogleNet, VGG16 and ResNet50 were implemented and their performance was compared using accuracy, precision, recall and F1-score.

Among the models tested, ResNet50 achieved the highest test accuracy of 79.77%, followed by VGG16 with 76.90% and GoogleNet with 73.37%. LeNet-5 gave the lowest accuracy of 62.04%, but it was also the fastest model to train. This shows the trade-off between model complexity, accuracy and computational cost.

The VGG16 transfer learning experiment also showed a clear improvement after fine tuning. The best validation accuracy increased from 65.12% with the frozen base to 77.40% after fine tuning. The hyperparameter experiments further showed that partial freezing gave the best test accuracy of 70.32% among the configurations tested.

Overall, ResNet50 gave the best classification performance, while GoogleNet provided a good balance between accuracy and the number of parameters. The results showed that selecting a CNN model is not only about choosing the one with the highest accuracy. Training time, model size and how well the pretrained features can adapt to the target dataset also need to be considered.

22. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, 2012.
3. Karen Simonyan and Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” 2014.
4. Christian Szegedy et al., “Going Deeper with Convolutions,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation.
8. Keras Documentation.
9. CIFAR-10 Dataset Documentation.